

Homework 12

张子乐, 3230104237

2026-05-26

1. Background

The MNIST data set contains grayscale images of handwritten digits. Each image has size 28×28 , so one image can be represented as a vector with $D = 784$ pixel values.

In this homework, the grayscale values are converted into **binary** values: pixels larger than a fixed threshold are set to 1, and the remaining pixels are set to 0.

First, we use the digits 3, 4, and 5 only. Although their labels are used to select the data, the mixture model itself is fitted without using the labels.

The initial goal is to estimate $K = 3$ latent Bernoulli components by the expectation-maximization (EM) algorithm and then reshape the estimated Bernoulli parameters back to images.

2. Model

Let $\mathbf{x}_n \in \{0, 1\}^D$ be the binarized and flattened image for observation n . A Bernoulli mixture model assumes that each observation is generated from one of K latent components:

$$p(\mathbf{x}_n) = \sum_{k=1}^K \pi_k p(\mathbf{x}_n | \mu_k),$$

where π_k is the mixing proportion of component k , $\sum_{k=1}^K \pi_k = 1$, and $\mu_k = (\mu_{k1}, \dots, \mu_{kD})$ contains the Bernoulli probabilities for the pixels in component k . The component likelihood is

$$p(\mathbf{x}_n | \mu_k) = \prod_{j=1}^D \mu_{kj}^{x_{nj}} (1 - \mu_{kj})^{1-x_{nj}}.$$

The log-likelihood is

$$\log p(X) = \sum_{n=1}^N \log \left[\sum_{k=1}^K \pi_k p(\mathbf{x}_n | \mu_k) \right].$$

3. Data Preparation and Helper Functions

3.1 Hyperparameters

- In the experiment below, the digit images are first converted into binary vectors by setting all pixels whose grayscale values exceed 127 to 1 and setting the remaining pixels to 0.
- We fit a data set containing the handwritten digits 3, 4, and 5, with at most 600 images sampled from each digit, using a mixture of $K = 3$ Bernoulli distributions.
- The EM algorithm is run for at most 100 iterations.
- The mixing coefficients are initialized as $\pi_k = 1/K$, and the Bernoulli parameters μ_{kj} are initialized independently from the uniform distribution $U(0.25, 0.75)$.

```
selected_digits <- c(3, 4, 5) # 选择 3,4,5 三个数字
K <- length(selected_digits)
max_per_digit <- 600 # 每个数字最多选取 600 个样本
threshold <- 127 # 灰度值二值化的阈值
max_iter <- 100 # EM 算法的最大迭代次数
tol <- 1e-6 # 收敛阈值
eps <- 1e-6
set.seed(42)
```

3.2 Helper Functions

```
# 数值稳定性
log_sum_exp <- function(log_values) {
  max_values <- apply(log_values, 1, max)
  max_values + log(rowSums(exp(log_values - max_values)))
}

clip_prob <- function(x, eps = 1e-6) {
  pmin(pmax(x, eps), 1 - eps)
}

image_to_df <- function(values) {
  image_matrix <- matrix(values, nrow = 28, ncol = 28, byrow = TRUE)
  df <- as.data.frame(as.table(image_matrix))
  names(df) <- c("row", "col", "value")
}
```

```

df$row <- as.integer(df$row)
df$col <- as.integer(df$col)
df$x <- df$col
df$y <- 29 - df$row
df
}

plot_digit_image <- function(values, title) {
  ggplot(image_to_df(values), aes(x = x, y = y, fill = value)) +
    geom_raster() +
    coord_fixed() +
    scale_fill_gradient(low = "white", high = "black", limits = c(0, 1)) +
    labs(title = title) +
    theme_void(base_size = 11) +
    theme(
      legend.position = "none",
      plot.title = element_text(hjust = 0.5, face = "bold")
    )
}

```

3.3 Data Loading and Reading

```

# 下载 MNIST 数据集
mnist_dir <- file.path("data", "mnist")
download_mnist <- function(dest_dir = mnist_dir) {
  dir.create(dest_dir, recursive = TRUE, showWarnings = FALSE)
  files <- c(
    images = "train-images-idx3-ubyte.gz",
    labels = "train-labels-idx1-ubyte.gz"
  )
  base_url <- "https://storage.googleapis.com/cvdf-datasets/mnist"
  for (file_name in files) {
    dest <- file.path(dest_dir, file_name)
    if (!file.exists(dest)) {
      url <- paste(base_url, file_name, sep = "/")
      tryCatch(
        download.file(url, destfile = dest, mode = "wb", quiet = TRUE),
        error = function(e) {

```

```

    stop(
      "Could not download MNIST automatically. ",
      "Please place train-images-idx3-ubyte.gz and ",
      "train-labels-idx1-ubyte.gz in ",
      normalizePath(dest_dir, winslash = "/", mustWork = FALSE),
      ". Original error: ",
      conditionMessage(e),
      call. = FALSE
    )
  }
}
}

paths <- file.path(dest_dir, unname(files))
names(paths) <- names(files)
invisible(paths)
}

# 读取 MNIST 图像和标签
read_mnist_images <- function(path) {
  con <- gzfile(path, "rb")
  on.exit(close(con))
  magic <- readBin(con, integer(), n = 1, size = 4, endian = "big")
  n_images <- readBin(con, integer(), n = 1, size = 4, endian = "big")
  n_rows <- readBin(con, integer(), n = 1, size = 4, endian = "big")
  n_cols <- readBin(con, integer(), n = 1, size = 4, endian = "big")
  if (magic != 2051 || n_rows != 28 || n_cols != 28) {
    stop("Unexpected MNIST image file format.", call. = FALSE)
  }
  pixels <- readBin(
    con,
    what = integer(),
    n = n_images * n_rows * n_cols,
    size = 1,
    signed = FALSE
  )
  matrix(pixels, nrow = n_images, byrow = TRUE)
}

```

```

read_mnist_labels <- function(path) {
  con <- gzfile(path, "rb")
  on.exit(close(con))
  magic <- readBin(con, integer(), n = 1, size = 4, endian = "big")
  n_labels <- readBin(con, integer(), n = 1, size = 4, endian = "big")
  if (magic != 2049) {
    stop("Unexpected MNIST label file format.", call. = FALSE)
  }
  readBin(con, what = integer(), n = n_labels, size = 1, signed = FALSE)
}

mnist_files <- download_mnist()
images <- read_mnist_images(mnist_files["images"])
labels <- read_mnist_labels(mnist_files["labels"])

selected_index <- unlist(lapply(selected_digits, function(digit) {
  candidates <- which(labels == digit)
  sample(candidates, min(max_per_digit, length(candidates)))
}))

selected_images <- images[selected_index, , drop = FALSE]
selected_labels <- labels[selected_index]
binary_images <- 1L * (selected_images > threshold)

```

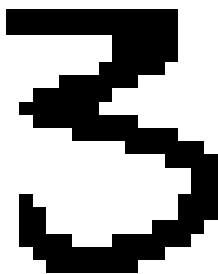
The following figure shows one binarized example from each selected digit.

```

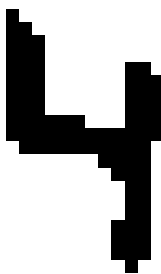
sample_plots <- lapply(selected_digits, function(digit) {
  first_index <- which(selected_labels == digit)[1]
  plot_digit_image(binary_images[first_index, ], paste("Digit", digit))
})
do.call(grid.arrange, c(sample_plots, nrow = 1))

```

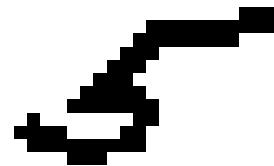
Digit 3



Digit 4



Digit 5



3.4 EM Algorithm Implementation

The latent variable z_{nk} indicates whether observation n belongs to component k . EM alternates between computing the posterior **responsibilities** and **updating the parameters**.

3.4.1 E-step

In the E-step, compute the responsibilities

$$\gamma_{nk} = \frac{\pi_k p(\mathbf{x}_n | \mu_k)}{\sum_{\ell=1}^K \pi_\ell p(\mathbf{x}_n | \mu_\ell)}$$

3.4.2 M-step

In the M-step, first define the effective sample size

$$N_k = \sum_{n=1}^N \gamma_{nk}$$

Then update

$$\pi_k = \frac{N_k}{N}, \quad \mu_{kj} = \frac{1}{N_k} \sum_{n=1}^N \gamma_{nk} x_{nj}$$

The computation below uses log probabilities for numerical stability.

```
EM <- function(x, K, max_iter = 100, tol = 1e-6, eps = 1e-6) {
  n <- nrow(x) # n 个样本
  d <- ncol(x) # d 维
```

```

# Initialize pi and mu
pi <- rep(1 / K, K)
mu <- matrix(runif(K * d, min = 0.25, max = 0.75), nrow = K, ncol = d)
log_likelihood <- numeric()
responsibilities <- matrix(NA_real_, nrow = n, ncol = K)

for (iter in seq_len(max_iter)) {
  mu <- clip_prob(mu, eps)
  pi <- clip_prob(pi, eps)
  pi <- pi / sum(pi)

  # E-step
  log_component <- x %*% t(log(mu)) + (1 - x) %*% t(log(1 - mu)) # log p(x_n | mu_k)
  log_joint <- sweep(log_component, 2, log(pi), "+") # 按列加上 log pi_k
  log_norm <- log_sum_exp(log_joint)
  responsibilities <- exp(log_joint - log_norm) # responsibilities

  current_ll <- sum(log_norm) # 当前的 log-likelihood
  log_likelihood <- c(log_likelihood, current_ll)

  # M-step
  nk <- colSums(responsibilities) # effective number of data points
  pi <- nk / n
  mu <- t(responsibilities) %*% x # K x D
  mu <- sweep(mu, 1, nk, "/")
  mu <- matrix(mu, nrow = K, ncol = d)

  # 判断是否收敛
  if (iter > 1) {
    improvement <- abs(log_likelihood[iter] - log_likelihood[iter - 1])
    scale <- abs(log_likelihood[iter - 1]) + eps
    if (improvement / scale < tol) {
      break
    }
  }
}

mu <- clip_prob(mu, eps)
pi <- clip_prob(pi, eps)

```

```

pi <- pi / sum(pi)
log_component <- x %*% t(log(mu)) +
  (1 - x) %*% t(log(1 - mu))
log_joint <- sweep(log_component, 2, log(pi), "+")
log_norm <- log_sum_exp(log_joint)
responsibilities <- exp(log_joint - log_norm)
final_log_likelihood <- sum(log_norm)

list(
  pi = pi,
  mu = clip_prob(mu, eps),
  responsibilities = responsibilities,
  log_likelihood = log_likelihood,
  final_log_likelihood = final_log_likelihood,
  iterations = length(log_likelihood)
)
}

```

4. Experiment Results

```

em_fit <- EM(
  x = binary_images,
  K = K,
  max_iter = max_iter,
  tol = tol,
  eps = eps
)
cat("Weights (pi):", round(em_fit$pi, 4), "\n")
Weights (pi): 0.3188 0.2924 0.3889

```

4.1 Convergence

The log-likelihood curve is used to check convergence.

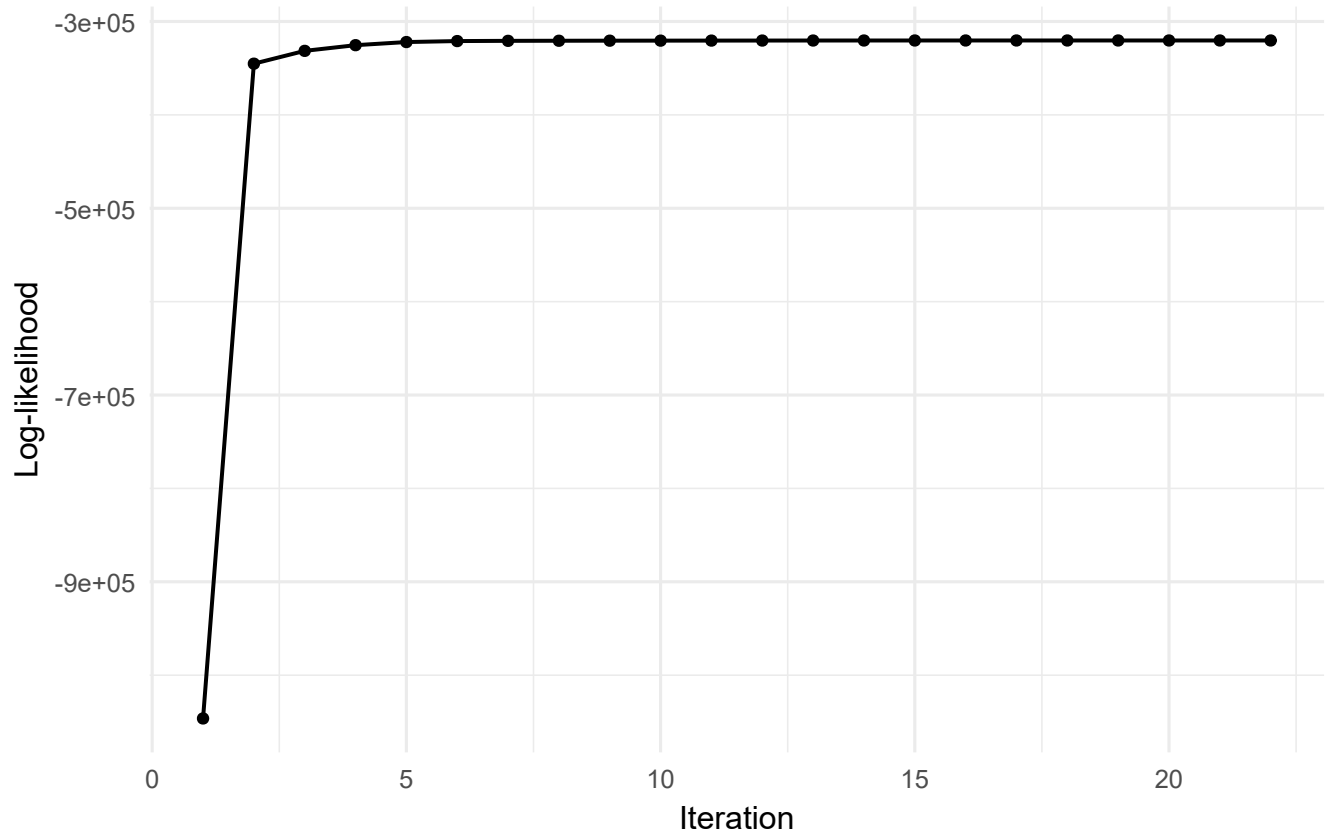
```

likelihood_df <- data.frame(
  iteration = seq_along(c(em_fit$log_likelihood, em_fit$final_log_likelihood)),
  log_likelihood = c(em_fit$log_likelihood, em_fit$final_log_likelihood)
)

```

```
)

ggplot(likelihood_df, aes(x = iteration, y = log_likelihood)) +
  geom_line(linewidth = 0.7, color = "#000000") +
  geom_point(size = 1.5, color = "#000000") +
  labs(
    x = "Iteration",
    y = "Log-likelihood"
  ) +
  theme_minimal(base_size = 12)
```

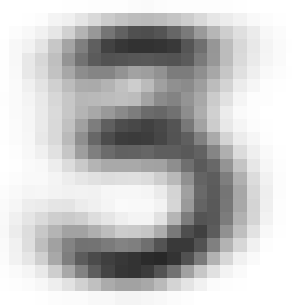
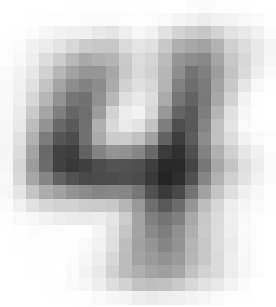
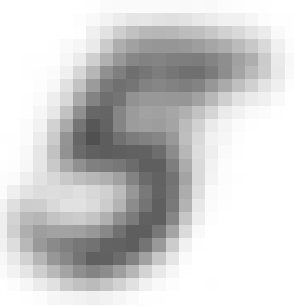


4.2 Reconstructed Digit Templates

Each row of μ is a vector of 784 Bernoulli probabilities. After reshaping it into a 28×28 matrix, dark pixels indicate locations that are estimated to be active with high probability under that component. We can distinguish the components by visually comparing the reconstructed templates with the shapes of the selected handwritten digits.

```
template_plots <- lapply(seq_len(K), function(k) {
  title <- paste0("Component ", k, ", pi = ", round(em_fit$pi[k], 3))
  plot_digit_image(em_fit$mu[k, ], title)
})
do.call(grid.arrange, c(template_plots, nrow = 1))
```

Component 1, pi = 0.319 Component 2, pi = 0.292 Component 3, pi = 0.389



4.3 Comment on Results

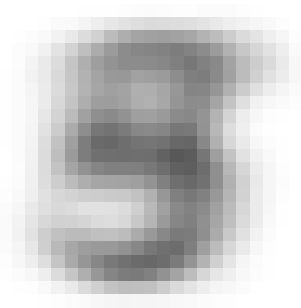
- The reconstructed templates suggest that Component 1 approximately corresponds to digit 5, Component 2 to digit 4, and Component 3 to digit 3.
- The ordering of these components is arbitrary because the EM algorithm is fitted without using the digit labels. Although the sampled data contain approximately equal numbers of digits 3, 4, and 5, the estimated mixture weights do not have to be exactly 1/3, because the latent components are inferred from image patterns rather than fixed by the observed labels.

4.4 Comparison with a Single Bernoulli Distribution

As a comparison, we also fit the same data set using a single multivariate Bernoulli distribution, again using maximum likelihood. This amounts to simply averaging the counts in each pixel, which produces a blurred average template, whereas the mixture model separates the major digit patterns into interpretable components.

```
single_mu <- colMeans(binary_images)
plot_digit_image(single_mu, "Single Bernoulli")
```

Single Bernoulli



5. Choosing the Number of Components

To investigate whether the number of latent components can be inferred without prespecifying K , we fitted Bernoulli mixture models with several candidate values of K and compared their AIC/BIC values.

For a Bernoulli mixture model with K components and D pixels, the number of free parameters is

$$p = KD + (K - 1),$$

where KD counts the Bernoulli probabilities and $K - 1$ counts the free mixing coefficients. Therefore,

$$\text{AIC} = 2p - 2\ell, \quad \text{BIC} = \log(N)p - 2\ell,$$

where ℓ is the maximized log-likelihood.

Each candidate value of K is fitted multiple times and the run with the largest final log-likelihood is retained and used to compute AIC/BIC for each K .

```
set.seed(4237)
candidate_K <- 1:10 # 评估 K=1 到 10 的模型
n_starts <- 3 # 重复运行 3 次 EM 算法, 选择对数似然最大的结果
n <- nrow(binary_images)
d <- ncol(binary_images)
best_models <- vector("list", length(candidate_K))
names(best_models) <- as.character(candidate_K)
model_selection_rows <- vector("list", length(candidate_K))
```

```

for (i in seq_along(candidate_K)) {
  k <- candidate_K[i]
  fits <- lapply(seq_len(n_starts), function(start) {
    EM(
      x = binary_images,
      K = k,
      max_iter = max_iter,
      tol = tol,
      eps = eps
    )
  })
  log_likeliheids <- vapply(
    fits,
    function(fit) fit$final_log_likeliheid,
    numeric(1)
  )
  best_start <- which.max(log_likeliheids)
  best_fit <- fits[[best_start]]
  best_models[[as.character(k)]] <- best_fit

  n_params <- k * d + (k - 1)
  log_likeliheid <- best_fit$final_log_likeliheid
  model_selection_rows[[i]] <- data.frame(
    K = k,
    best_start = best_start,
    log_likeliheid = log_likeliheid,
    parameters = n_params,
    AIC = 2 * n_params - 2 * log_likeliheid,
    BIC = log(n) * n_params - 2 * log_likeliheid,
    iterations = best_fit$iterations
  )
}

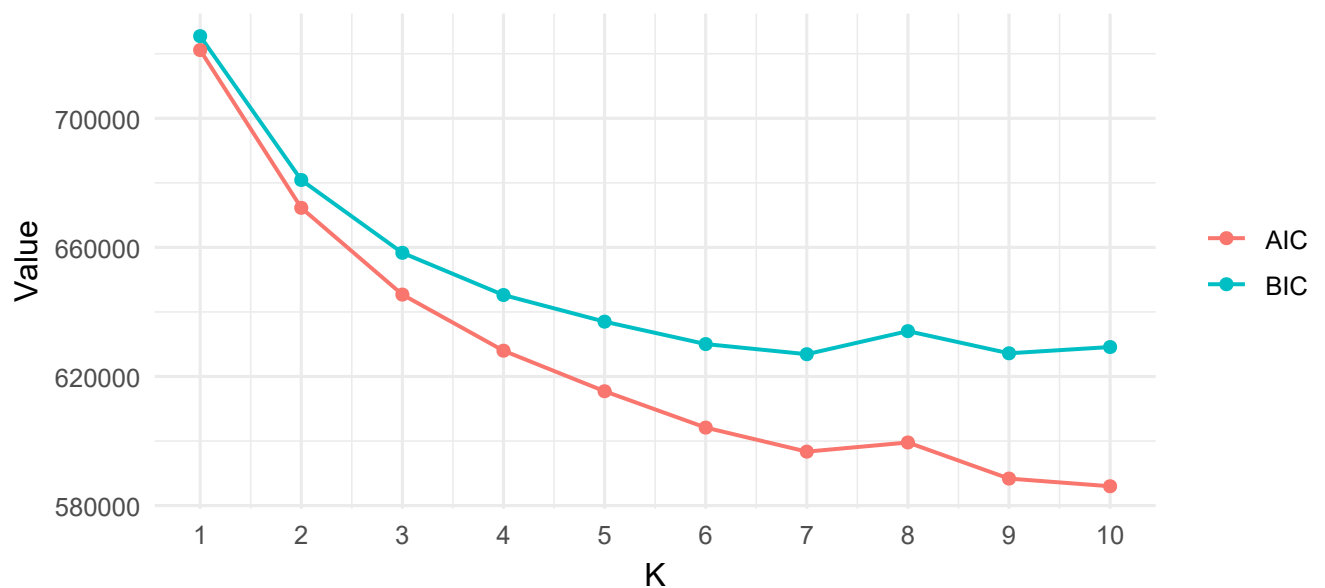
model_selection <- do.call(rbind, model_selection_rows)
# 不同 K 下的对数似然/AIC/BIC
model_selection

```

	K	best_start	log_likeliheid	parameters	AIC	BIC	iterations
1	1	1	-359787.8	784	721143.7	725452.2	3
2	2	3	-334559.9	1569	672257.9	680880.4	27

3	3	3	-320335.7	2354	645379.5	658316.0	42
4	4	2	-310857.8	3139	627993.5	645244.0	59
5	5	2	-303790.2	3924	615428.4	636992.9	51
6	6	1	-297374.2	4709	604166.5	630045.0	35
7	7	1	-292863.6	5494	596715.2	626907.7	93
8	8	3	-293495.7	6279	599549.4	634055.9	63
9	9	2	-287121.1	7064	588370.2	627190.7	43
10	10	3	-285146.2	7849	585990.5	629125.0	71

```
selection_plot_df <- rbind(
  data.frame(K = model_selection$K, criterion = "AIC", value = model_selection$AIC),
  data.frame(K = model_selection$K, criterion = "BIC", value = model_selection$BIC)
)
ggplot(selection_plot_df, aes(x = K, y = value, color = criterion)) +
  geom_line(linewidth = 0.7) +
  geom_point(size = 1.8) +
  scale_x_continuous(breaks = candidate_K) +
  labs(
    x = "K",
    y = "Value",
    color = NULL
  ) +
  theme_minimal(base_size = 12)
```



```
# 选择使 AIC/BIC 最小的 K
```

```
k_AIC <- model_selection$K[which.min(model_selection$AIC)]
```

```
k_BIC <- model_selection$K[which.min(model_selection$BIC)]
```

```
cat("Selected K by AIC:", k_AIC, "\n")
```

```
Selected K by AIC: 10
```

```
cat("Selected K by BIC:", k_BIC, "\n")
```

```
Selected K by BIC: 7
```

```
em_AIC <- best_models[[as.character(k_AIC)]]
```

```
em_BIC <- best_models[[as.character(k_BIC)]]
```

```
cat("Weights (pi) for AIC:", round(em_AIC$pi, 4), "\n")
```

```
Weights (pi) for AIC: 0.0965 0.0576 0.1192 0.1117 0.1444 0.0367 0.1237 0.086 0.1119 0.1123
```

```
cat("Weights (pi) for BIC:", round(em_BIC$pi, 4), "\n")
```

```
Weights (pi) for BIC: 0.1932 0.1494 0.1469 0.1134 0.1252 0.1363 0.1358
```

```
template_plots_BIC <- lapply(seq_len(k_BIC), function(k) {
```

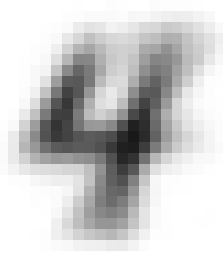
```
  title <- paste0("pi = ", round(em_BIC$pi[k], 3))
```

```
  plot_digit_image(em_BIC$mu[k, ], title)
```

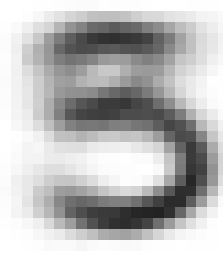
```
})
```

```
do.call(grid.arrange, c(template_plots_BIC, nrow = 2))
```

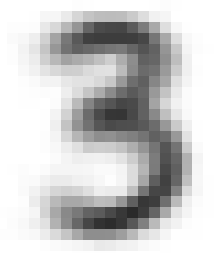
pi = 0.193



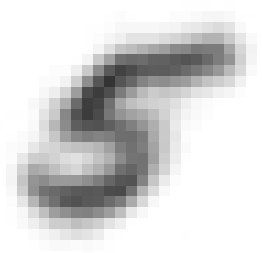
pi = 0.149



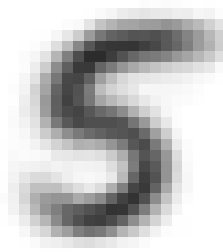
pi = 0.147



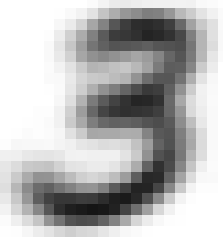
pi = 0.113



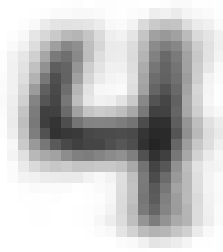
pi = 0.125



pi = 0.136



pi = 0.136



Conclusion

- The AIC reaches its minimum at $K = 10$, whereas the BIC reaches its minimum at $K = 7$. Each value of K is fitted with 3 random starts, and the AIC/BIC values are computed from the run with the largest log-likelihood.
- Although the data contain only three digit labels, the number of mixture components does not need to equal three. Different handwriting styles within the same digit may be captured by separate components.
- The saved best model for $K = 7$ is plotted above. Some components represent different styles of the same digit, such as upright and slanted forms.
- Therefore, the additional components should be interpreted as handwriting-style subtypes rather than new digit classes.